



Qualcomm Technologies, Inc.

00034.1 Release Note for SW5100.LAW.2.0

Release Notes

SW5100.LAW.2.0

RNO-250929191636 Rev. 1

September 29, 2025

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

Confidential - Qualcomm Technologies, Inc. and/or its affiliated companies - May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to:
DocCtrlAgent@qualcomm.com.

© Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Table Of Contents

Download.....	4
Release History.....	5
Getting Started.....	6
Software maturity of SP.....	6
Download QTI proprietary software from Qualcomm ChipCode portal.....	6
Android Build Informations:	7
Build Instructions.....	9
Metabuild components:.....	9
Download.....	9
.....	9
and.....	9
.....	9
Build HLOS software images	9
Sample Commands to download & generate different workspace combinations.....	9
Build non-HLOS software images	12
Generat.....	27
build.....	27
Generate QCC5100 (NOR Flash) META build and Sideload.....	27
Program CDT.....	29
Memory configuration.....	30
Supported ASICs.....	30
.....	30
ASICs no longer supported.....	30
Platform Information.....	30
Memory Platform Information.....	31
RF Card.....	31
Memory Map Info (MSM):.....	31
Memory Map Info (APQ):.....	34
Memory Map Info (QCC5100):.....	37
.....	41
New Features.....	42
Limitations.....	43
Issues.....	43
New Issues	43
Unresolved Issues.....	43
Resolved.....	43
Limitation.....	43

Test Reports.....	44
IDP Nor Variant.....	44
WDP Nor Variant.....	45
NOFLASH_PSRAM Variant.....	47
APQ Variant.....	48
References.....	51

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

1. Download

A. ChipCode Location:

https://chipcode.qti.qualcomm.com/38633/sw5100-law-2-0_ap_oem_osh

https://chipcode.qti.qualcomm.com/38633/sw5100-law-2-0_hlos_device_apq-oss

https://chipcode.qti.qualcomm.com/38633/sw5100-law-2-0_ap_standard_oem

B. ChipCode Commands:

git clone --depth 1 <repo location>

Please note that the <repo location> and clone path can be obtained from the "Files" tab in the "Clone this repository" section of the above chipcode location.

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

2. Release History

ChipCode Release History:

https://createpoint.qti.qualcomm.com/planner/link/search/newReleaseHistory/preset/sw5100-law-2-0_ap_oem_osh/master/r00034.1

https://createpoint.qti.qualcomm.com/planner/link/search/newReleaseHistory/preset/sw5100-law-2-0_hlos_device_apq-oss/master/r00034.1

https://createpoint.qti.qualcomm.com/planner/link/search/newReleaseHistory/preset/sw5100-law-2-0_ap_standard_oem/master/r00034.1

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

3. Getting Started

Software maturity of SP

Software product	Maturity level
SW5100.LAW.2.0	POST-CS4

Download QTI proprietary software from Qualcomm ChipCode portal

Qualcomm Technologies, Inc. (QTI) software can be downloaded from the Qualcomm ChipCode website. Designated points of contact in your organization can download the licensed software. The software is organized into distribution packages (distros) composed of subsystem image files. Each distro has a corresponding Git project. The Git tree includes revisions for previous builds that allow you to diff the changes between releases.

If you are accessing Qualcomm ChipCode for the first time, QTI recommends a review of the up-to-date documentation and tutorial videos at <https://chipcode.qti.qualcomm.com/helpki>.

Follow these steps to download the QTI proprietary software.

1.Go to the ChipCodeTM portal by clicking the link: <https://chipcode.qti.qualcomm.com>

2.Select <SW5100.LAW.2.0> product from the Products list.

A list of distributions (distros) available to the company is displayed.

In general, the sw5100-law-2-0_amss_standard_oem distro contains the source code .

3.Choose a distro to see the available release tags.

The release tag closely matches the build ID in the release notes. For this release:

- Build ID: POST-CS4 0.0.034.1
- Chipcode tag: r00034.1

4. Download the distro from the Qualcomm ChipCode website by using one of the following methods:

Note: The about.html and contents.xml files contain the build ID used to download the open-source software from CodeLinaro sites

- Method 1:
 - On the distro page, select a release tag and use the Download Release button to download in GZip (Linux) or Zip (Windows) format.
 - Create a top-level local directory on the build PC and decompress the downloaded

software package.

- Method 2: On the distro page, choose the <release-tag> execute the command in the Clone this release box to clone the release with git, and then check out the specific release tag.
 - Following is an example of the git commands:
 - NOTE: A shallow clone copies only the specific release. A full clone is performed to copy all available releases, but may result in large transfers and possible connection drops.
 - Shallow clone
\$ git clone -b <release-tag> --depth 1
https://chipmaster2.qti.qualcomm.com/home2/git/qualcomm/<sp-release>.git
 - Full clone
\$ git clone -b <release-tag>
https://chipmaster2.qti.qualcomm.com/home2/git/qualcomm/<sp-release>.git

Request customer access to CodeLinaro (CLO)

Private CLO access requires a CLO account.

- One time CLO set-up is required, see “Set up CodeLinaro” section in the Quick Start Guide (80-35346-1).
- If one does not have a personal access token, generate one by following instructions in “Generate personal access tokens”.

For this SP, we are public CLO:

- clo/la/la/vendor/manifest
- clo/la/la/system/manifest
- clo/la/kernelplatform/manifest

Android Build Informations:

Date	Tag/ BuildId	Chipset	Android Version
Sept 30, 2025	VENDOR: AU_LINUX_ANDROID_LAW.VENDOR.14.0.R1.11.00.00.1139.112 QWSI 14.0: ÄU_LINUX_ANDROID_LAW.QWSI.14.0.R1.11.00.00.1143.050 KERNEL: AU_LINUX_KERNEL.PLATFORM.2.2.R1.00.00.00.054.048	SW5100	Android U

QKI TAG:

Android Upstream Tag: UPM1.231025.053

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

4. Build Instructions

Metabuild components:

Software Image	Build ID
Meta	SW5100.LAW.2.0-00034-STD.PROD-1
ADSP.IT.2.1	ADSP.IT.2.1-00165.1-ATHERTON-1
APPS.SLATE.3.0	APPS.SLATE.3.0-02249-STD.PROD-1
BOOT.XF.4.3	BOOT.XF.4.3-00258-ATHERTONLAZ-3
BTFW.SLATE.2.0.3	BTFW.SLATE.2.0.3-00032-PATCHZ-1
DSP.SLATE.1.1	DSP.SLATE.1.1-00150-ATHERTON-1
BTFM.CHE.2.1.7	BTFM.CHE.2.1.7-00059-QCACHROMZ-1
KERNEL.PLATFORM.2.2.r1	KERNEL.PLATFORM.2.2.r1-04800-kernel.0-3
LAW.QWSI.14.0.r1	LAW.QWSI.14.0.r1-05000-qssi.0-1
LAW.VENDOR.14.0.r1	LAW.VENDOR.14.0.r1-11200-SW5100.0-1
MPSS.HA.2.0	MPSS.HA.2.0-00524-ATHERTON_GEN_PACK-1
MPSS.HA.2.0.c2	MPSS.HA.2.0.c2-00038-ATHERTON_GEN_PACK-1
RPM.BF.1.12	RPM.BF.1.12-00110-ATHERTONAAAAANAZR-1
TMEL.SLATE.2.0	TMEL.SLATE.2.0-00204-SLATE-2
TZ.APPS.2.0.1	TZ.APPS.2.0.1-00142-KAMORTAAAAANAZT-4
TZ.XF.5.1.5	TZ.XF.5.1.5-00358-ATHERTONAAAAANBZT-1
WLAN.HL.3.4.1.C1	WLAN.HL.3.4.1.c1-00248-QCAHLSWMTPLZ-1

For the build components and IDs, refer to the [about.html](#) file in the root of the repository on Qualcomm ChipCode

4.1 Download and Build HLOS software images

Prerequisites:

- The HLOS proprietary code is downloaded from the ChipCode portal.

4.1.1 Sample Commands to download & generate different workspace combinations

VENDOR TREE (CLO + Proprietary content from Qualcomm ChipCode with History):

```
$ mkdir -p <law.vendor.14.0 workspace path>
```

```
$ cd <law.vendor.14.0 workspace path>
```

```

$ cp <absoulte path where chipcode extracted>/LAW.VENDOR.14.0/LINUX/android/sync_snap_v2.sh ./
$ ./sync_snap_v2.sh \
--
workspace_path=/local/mnt/workspace/tmp/roboticsint/product/sw5100.law.2.0/LAW.VENDOR.14.0.r1
\
--image_type=la \
--tree_type=la_tree \
--prop_opt=chipcode \
--common_oss_url=https://git.codelinaro.org \
--vendor_oss_manifest_git=clo/la/la/vendor/manifest \
--vendor_au_tag=AU_LINUX_ANDROID_LAW.VENDOR.14.0.R1.11.00.00.1139.112 \
--vendor_chipcode_path=<absoulte path where chipcode
extracted>/LAW.VENDOR.14.0/LINUX/android \
--qssi_oss_manifest_git=clo/la/la/system/manifest \
--qssi_chipcode_path=<absoulte path where chipcode extracted>/LAW.QWSI.14.0/LINUX/android \
--kernel_oss_manifest_git=clo/la/kernelplatform/manifest \
--kernel_au_tag=AU_LINUX_KERNEL.PLATFORM.2.2.R1.00.00.00.054.048 \
--kernel_chipcode_path=<absoulte path where chipcode
extracted>/KERNEL.PLATFORM.2.2/kernel_platform \
--repo_url=https://git.codelinaro.org/clo/tools/repo.git \
--repo_branch=aosp-new/stable \
--shallow_clone=true

```

For Compilation of Vendor:

```

$ source build/envsetup.sh
$ lunch monaco_go_aon-userdebug
$ chmod 777 ./kernel_platform/build/android/prepare_vendor.sh
$ chmod 777 kernel_platform/build/build_module.sh
$ chmod 777 kernel_platform/build/android/merge_dtbs.sh
$ ln -sn ../../../../qcom/proprietary/devicetree/ kernel_platform/msm-
kernel/arch/arm64/boot/dts/vendor
$ export TARGET_BOARD_PLATFORM=monaco
$ RECOMPILE_KERNEL=1 LTO=thin ./kernel_platform/build/android/prepare_vendor.sh monaco
consolidate
$ chmod 777 ./build.sh
$ ./build.sh -j8 dist --target_only

```

LAW.QWSI.14 STANDALONE TREE (CLO + Proprietary content from Qualcomm ChipCode with History):

```

$ mkdir -p <law.qwsi.14.0 workpace path>
$ cd <law.qwsi.14.0 workpace path>

```

```
$ cp <absoulte path where chipcode extracted>/LAW.VENDOR.14.0/LINUX/android/sync_snap_v2.sh ./
$ ./sync_snap_v2.sh \
--workspace_path=<law.qwsi.14.0 workpace path> \
--image_type=la \
--tree_type=la_qssi \
--prop_opt=chipcode \
--common_oss_url=https://git.codelinaro.org \
--qssi_oss_manifest_git=clo/la/la/system/manifest \
--qssi_au_tag=AU_LINUX_ANDROID_LAW.QWSI.14.0.R1.11.00.00.1143.050 \
--qssi_chipcode_path=<absoulte path where chipcode extracted>/LAW.QWSI.14.0/LINUX/android \
--repo_url=https://git.codelinaro.org/clo/tools/repo.git \
--repo_branch=aosp-new/stable \
--shallow_clone=true
```

For Compilation of QWSI 14:

```
$ source build/envsetup.sh
$ lunch qssi_32go-userdebug
$ bash build.sh -j32 dist --qssi_only
```

Generate super.img

```
$ python vendor/qcom/opensource/core-utils/build/build_image_standalone.py --image super --
qssi_build_path <law.qwsi.14.0 workpace path> --target_build_path <law.vendor.14.0 workpace path>
--merged_build_path <law.vendor.14.0 workpace path> --target_lunch monaco_go_aon --output_ota
```

KERNEL.PLATFORM STANDALONE TREE (CLO + Proprietary content from Qualcomm ChipCode with History):

```
$ mkdir -p <kernel_workspace_path>
$ cd <kernel_workspace_path>
$ ./sync_snap_v2.sh \
--image_type=la \
--tree_type=kernel_platform \
--prop_opt=chipcode \
--workspace_path=<kernel_workspace_path> \
--common_oss_url=https://git.codelinaro.org \
--kernel_oss_manifest_git=clo/la/kernelplatform/manifest \
--kernel_au_tag=AU_LINUX_KERNEL.PLATFORM.2.2.R1.00.00.00.054.048 \
--kernel_chipcode_path=<absoulte path where chipcode
extracted>/KERNEL.PLATFORM.2.2/kernel_platform \
--repo_url=https://git.codelinaro.org/clo/tools/repo.git \
--repo_branch=aosp-new/stable \
--shallow_clone=true
```

For Compilation of KERNEL:

```
$ cd <kernel_workspace_path>  
$ ln -sn ../../../../qcom/proprietary/devicetree/ kernel_platform/msm-  
kernel/arch/arm64/boot/dts/vendor  
$ cd kernel_platform  
$ ./build_with_bazel.py -t monaco consolidate --lto=thin
```

4.2 Build non-HLOS software images

4.2.1 Toolchain

The tables list the toolchain and version dependencies for individual non-HLOS components, which should be installed on the build machine.

Table 5-1 Non-HLOS toolchain to build source for SW5100.LAW.2.0

Qualcomm
Confidential - May Contain Trade Secrets
2026-06-16 10:36:48 GMT
arumugaraj@titan.co.in

Component build release	Source or binary only	Toolchain required for building source	Python version	Perl version	Cygwin	Supported build hosts
HLOS	Source	-	-	-	-	Ubuntu 16.04 and later
MPSS	Source	Hexagon 8.3.08	Python 3.8.2	Perl 5.18.2 Linux builds only	Version 2.831	Ubuntu 16 and later, Windows 7
aDSP	Source	Hexagon 8.3.12	Python 3.8.2	-	-	Ubuntu 16 and later, Windows 7
XBL (boot build)	Source	◦ Snapdragon LLVM Arm compiler 10.0.3 ◦ Linaro AARCH64 GCC <i>gcc-linaro-aarch64-none-elf-4.9-2014.07</i>	Python 3.8.2	Perl 5.8.x Linux builds only	Windows builds only; Needs tee.exe	Ubuntu 16.04 and later, Windows 10
RPM	Source	Snapdragon LLVM Arm compiler 3.9.2	Python 3.8.2	-	Windows builds only; needs tee.exe	Ubuntu 16 and later, Windows 7 only
TZ	Binary	◦ Snapdragon LLVM Arm compiler 8.0.9 ◦ Linaro AARCH64 GCC <i>gcc-linaro-aarch64-none-elf-4.9-2014.07</i>	Python 3.8.2	-	-	-
WLAN	Binary	-	-	-	-	-
VENUS	Binary	-	-	-	-	-
APPS.SLATE	Source	Arm Compiler 6.14.1	Python 3.7 or later	-	-	Ubuntu 16.04, Windows 10
DSP.SLATE	Source	Xtensa_2021.7 HiFi5_slate_v1_1	Python 3.8	-	-	Ubuntu 16 and later, Windows 10

To apply patches or configurations, update folder structure, update compilation steps, and so on, see Release Note for SW5100.LAW.2.0.

4.2.2 Set build environment

The reference commands enable you to set environment variables for toolchains of all non-HLOS components. Not using these environment variables does not impact the build. The environment variables specific to a non-HLOS component can be set. For information on tool dependency and version, see Table 5-1 Non-HLOS toolchain to build source for SW5100.LAW.2.0. Accordingly, set the required environment variable.

Ensure that `PYTHON_PATH` and `SECTOOLS` variables are set for all the non-HLOS components.

- **Windows build environment**

```
set PYTHON_PATH=<path_to_python_3.8.x>\bin
```

```
set PATH=%PATH%;<path_to_llvm_arm_compiler>\bin
```

```
set ARMLMD_LICENSE_FILE=<mylicense_file>@<mylicense_server>
```

```
set ARMTTOOLS=LLVM
```

```
set HEXAGON_ROOT=<path_to_hexagon_tool_install_folder>
```

```
set SECTOOLS=<target_root>\SW5100.LAW.2.0\common\sectoolsv2\ext\Windows\sectools.exe
```

- **Linux build environment**

```
export PYTHON_PATH=<path_to_python_3.8.x>/bin
```

```
export ARMLMD_LICENSE_FILE=<mylicense_file>@<mylicense_server>
```

```
export ARMTTOOLS=LLVM
```

```
export HEXAGON_ROOT=<path_to_hexagon_tool_install_folder>
```

```
export SECTOOLS=<target_root>/SW5100.LAW.2.0/common/sectoolsv2/ext/Linux/sectools
```

4.2.3 Build MPSS

For APQ distro **sw5100-law-2-0_ap_standard_oem**, the MPSS(MPSS.HA.2.0.C2) compilation is not required .

For AMSS distro **sw5100-law-2-0_amss_standard_oem**, you can follow the steps below to build the MPSS (MPSS.HA.2.0) code.

Set environment variables for MPSS build. For more information, see Set build environment.

Setup nanopb

nanopb is automatically downloaded as part of the MPSS build. If there is an error downloading nanopb at build time, then nanopb can be downloaded manually.

Windows build environment

Integrating and setting up nanopb dependency with modem build is required once only for the modem image.

1. Download **nanopb-0.3.9.5-windows-x86.zip** from <https://jpa.kapsi.fi/nanopb/download/nanopb-0.3.9.5-windows-x86.zip>
2. Copy the **nanopb-0.3.9.5-windows-x86.zip** file to:
`<target_root>\MPSS.HA.2.0\modem_proc\ssc_api.`
3. Open a command terminal, and go to directory: `cd <target_root>\MPSS.HA.2.0\modem_proc\`
4. Run the following command:

```
python ssc_api\build\config_nanopb_dependency.py -f nanopb-0.3.9.5-windows-x86
```

The output of this command is similar to the following:

```
ssc_api\nanopb\inc
```

```
pb_encode.h
```

```
pb_common.h
```

```
pb.h
```

```
pb_decode.h
```

```
ssc_api\nanopb\src
```

```
pb_decode.c
```

```
pb_encode.c
```

```
pb_common.c
```

Linux build environment

1. Download **nanopb-0.3.9.5-linux-x86.tar.gz** from <https://jpa.kapsi.fi/nanopb/download/nanopb-0.3.9.5-linux-x86.tar.gz>
2. Copy **nanopb-0.3.9.5-linux-x86.tar.gz** to:
`<target_root>/MPSS.HA.2.0/modem_proc/ssc_api`
3. Ensure that all the contents in the folder (`<target_root>/MPSS.HA.2.0/modem_proc/ssc_api`) have both read and write permission
4. Open a command terminal, and cd to directory: `cd <target_root>/MPSS.HA.2.0/modem_proc/ssc_api`
5. Run the following command:

```
python ssc_api/build/config_nanopb_dependency.py -f nanopb-0.3.9.5-linux-x86
```

The output of this command is similar to the following:

ssc_api/nanopb-0.3.9.5-linux-x86.tar.gz

ssc_api/nanopb/nanopb-0.3.9.5-linux-x86

ssc_api/nanopb/inc

pb_encode.h

pb_common.h

pb.h

pb_decode.h

ssc_api/nanopb/src

pb_decode.c

pb_encode.c

pb_common.c

6. Go to ms directory:

cd <target_root>/MPSS.HA.2.0/modem_proc/build/ms

7. Based on the build environment and modem build flavor, run the appropriate build commands from the table:

Build flavors according to modem configuration:

- 3925 RF: atherton.genw3k.prod
- APQ RF: atherton.gps.prod
- LTEONLY RF: atherton.lteonly.prod

Atherton.genw3k.prod flavor is the default tested variant. Depending on your requirements, you can choose a different flavor.

Build commands: replace <build_flavor> with atherton.genw3k.prod or atherton.gps.prod or lteonly.prod.

Build environment	Build commands
Linux/Windows	◦ Build images: python ./build_variant.py <build_flavor> bparams=-k ◦ Clean the build: python ./build_variant.py <build_flavor> --clean

4.2.3 Build boot loader

Set environment variables for boot build. For more information, see Set build environment. Toolchain path can be updated in the file "BOOT.XF.4.3/boot_images/QcomPkg/toolchainconfig.json".

1. Go to the following directory: `cd <target_root>/BOOT.XF.4.3/boot_images/QcomPkg/SocPkg/AthertonPkg`

Where <target_root> is the top-level directory created in Download QTI proprietary software

2. Run build command from the table

see

`<target_root>/BOOT.XF.4.3/boot_images/QcomPkg/Docs/HowToBuildReadme.txt` for advanced build commands.

Build environment	Build command
Linux/Windows	◦ Build images python ../../buildex.py --variant LAB -r RELEASE -t AthertonPkg,QcomToolsPkg ◦ Clean build python ../../buildex.py --variant LAB -r RELEASE -t AthertonPkg,QcomToolsPkg --build_flags=cleanall

4.2.4 Build TZ and hypervisor

Set environment variables for TZ build. For more information, see Set build environment

1. Go to the following directory

`cd <target_root>/TZ.XF.5.1.5/trustzone_images/build/ms`

2. Run build command from the table to build all images. The TZ and hypervisor images are released as binaries. Sampleapp and devcfg are generated with the following commands.

Build environment	Build command
Linux/Windows	- Build images: python build_all.py -b TZ.XF.5.1 CHIPSET=atherton_1_5gb_ddr --config=build_config_deploy.xml --recompile - Clean the build: python build_all.py -b TZ.XF.5.1 CHIPSET=atherton_1_5gb_ddr --config=build_config_deploy.xml --clean

4.2.5 Build RPM

Set environment variables for RPM build. For more information, see Set build environment.

1. Go to the following directory

```
cd <target_root>/RPM.BF.1.12/rpm_proc/build
```

2. Based on the build environment, run the appropriate build commands from the table:

Build environment	Build command
Linux	- Build images: ./build_atherton.sh - Clean the build ./build_atherton.sh -c
Windows	- Build images: build_atherton.bat - Clean the build build_atherton.bat -c

4.2.6 Build ADSP

Set environment variables for ADSP build. For more information, see Set build environment.

Setup nanopb

nanopb is automatically downloaded as part of ADSP build process. If there is an issue downloading nanopb at build time, then nanopb can be downloaded manually.

Windows build environment

1. Download **nanopb-0.3.9.5-windows-x86.zip** from <https://jpa.kapsi.fi/nanopb/download/nanopb-0.3.9.5-windows-x86.zip>
2. Copy the **nanopb-0.3.9.5-windows-x86.zip** file to: <target_root>\ADSP.IT.2.1\adsp_proc\ssc_api
3. Open a command terminal, and go to directory: cd <target_root>\ADSP.IT.2.1\adsp_proc\
4. Run the following command:

```
python ssc_api\build\config_nanopb_dependency.py -f nanopb-0.3.9.5-  
windows-x86
```

The output of this command is similar to the following:

```
ssc_api\nanopb\inc
```

```
pb_encode.h
```

```
pb_common.h
```

```
pb.h
```

```
pb_decode.h
```

```
ssc_api\nanopb\src
```

```
pb_decode.c
```

```
pb_encode.c
```

```
pb_common.c
```

Linux build environment

1. Download **nanopb-0.3.9.5-linux-x86.tar.gz** from <https://jpa.kapsi.fi/nanopb/download/nanopb-0.3.9.5-linux-x86.tar.gz>
2. Copy **nanopb-0.3.9.5-linux-x86.tar.gz** to: <target_root>/ADSP.IT.2.1/adsp_proc/ssc_api
3. Ensure that all the contents in the folder (<target_root>/ADSP.IT.2.1/adsp_proc/ssc_api) have both read and write permission
4. Open a command terminal, and cd to directory: cd <target_root>/ADSP.IT.2.1/adsp_proc/ssc_api
5. Run the following command:

```
python ssc_api/build/config_nanopb_dependency.py -f nanopb-0.3.9.5-linux-x86
```

The output of this command is similar to the following:

```
ssc_api/nanopb-0.3.9.5-linux-x86.tar.gz
```

```
ssc_api/nanopb/nanopb-0.3.9.5-linux-x86
```

```
ssc_api/nanopb/inc
```

```
pb_encode.h
```

```
pb_common.h
```

```
pb.h
```

pb_decode.h

ssc_api/nanopb/src

pb_decode.c

pb_encode.c

pb_common.c

6. cd <target_root>/ADSP.IT.2.1/adsp_proc/build/ms

7. Run build command from the table

Build environment	Build command
Linux/Windows	For SW5100 + QCC5100 based platform: • Build images: python ./build_variant.py atherton.adsp_slate.prod python ./build_variant.py atherton.adsp_oda.prod • Clean the build python ./build_variant.py atherton.adsp_slate.prod -clean python ./build_variant.py atherton.adsp_oda.prod -clean

4.2.7 Build QCC5100 APSS and audio subsystem

4.2.7.1 Build application processor image

The ARMTTOOLS environment variable has been set to LLVM in previous step, so you need to unset it first (For example, in Linux, use: 'unset ARMTTOOLS').

The nanopb dependency should first be integrated before building the QCC5100 image.

Windows build environment

To integrate and set up nanopb dependency with APPS.SLATE build (one-time required for APPS.SLATE image) when compiling Build on Windows, perform these steps:

1. Download nanopb-0.3.9.5-windows-x86.zip from <https://jpa.kapsi.fi/nanopb/download/>.
2. Copy the nanopb-0.3.9.5-windows-x86.zip file to
<target_root>\APPS.SLATE.3.0\apps_proc\fw\services\sensors\ssc_api.
3. Open a command terminal, and cd to directory:
<target_root>\APPS.SLATE.3.0\apps_proc\fw\services\sensors\
4. Run the following command:

```
python ssc_api\build\config_nanopb_dependency.py -f nanopb-0.3.9.5-windows-x86
```

The output of this command is similar to the following:

ssc_api\nanopb\inc

pb_encode.h

pb_common.h

pb.h

pb_decode.h

ssc_api\nanopb\src

pb_decode.c

pb_encode.c

pb_common.c

Linux build environment

1. Download **nanopb-0.3.9.5-linux-x86.tar.gz** from <https://jpa.kapsi.fi/nanopb/download/>.
2. Copy the nanopb-0.3.9.5-linux-x86.tar.gz file to
<target_root>/APPS.SLATE.3.0/apps_proc/fw/services/sensors/ssc_api.
3. Open a command terminal, and cd to directory:
<target_root>/APPS.SLATE.3.0/apps_proc/fw/services/sensors
4. Run the following command:

```
python ssc_api/build/config_nanopb_dependency.py -f nanopb-0.3.9.5-  
linux-x86
```

The output of this command is similar to the following:

ssc_api/nanopb-0.3.9.5-linux-x86.tar.gz

ssc_api/nanopb/nanopb-0.3.9.5-linux-x86

ssc_api/nanopb/inc

pb_encode.h

pb_common.h

pb.h

pb_decode.h

ssc_api/nanopb/src

pb_decode.c

pb_encode.c

pb_common.c

5. Open a command prompt and navigate to the following directory:

```
cd <target root>/APPS.SLATE.3.0/apps_proc/build/bsp/
```

6. Run the build commands

WDP MEP NOR flash is the default tested variant.

Depending on your requirements, you can choose a different flavor.

- For WDP MEP Nor FLASH variant:

Build environment	Build command
Linux	◦ Build images: ./build.sh --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Clean and build: ./build.sh -c --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Build Ux offload enabled: ./build.sh --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors --widget_ux_offload
Windows	◦ Build images: Build.bat --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Clean and build: Build.bat -c --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Build Ux offload enabled: build.bat --norflash_psrpm --enable_audio --enable_disp_host --enable_ssc_sensors --widget_ux_offload

- For IDP MEP Nor FLASH variant:

Build environment	Build command
Linux	◦ Build images: ./build.sh --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Clean and build: ./build.sh -c --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Build Ux offload enabled: ./build.sh --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors --widget_ux_offload
Windows	◦ Build images: build.bat --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Clean and build: build.bat -c --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors ◦ Build Ux offload enabled: build.bat --norflash_h3v2psram --enable_audio --enable_disp_host --enable_ssc_sensors --widget_ux_offload

- For **NO Flash PSRAM** variant

- Navigate to SW5100.LAW.2.0/contents.xml and make the below changes

```
•
<!-- delete the below lines -->
    <file_ref slate_bin_apps="true">
<file_name>QCC5100_ARNFRIM_nonsecure_flash.elf</file_name>
    <file_path>apps_proc/build/ms/</file_path>
    </file_ref>
    <file_ref slate_bin_apps="true" slate_bin="true">
    <file_name>rwfs.bin</file_name>
    <file_path>apps_proc/build/ms/</file_path>
```

```

        </file_ref>
        <file_ref slate_bin_apps="true" slate_bin="true">
            <file_name>rofs0.bin</file_name>
<file_path>apps_proc/fw/services/audio/data/</file_path>
        </file_ref>
        <file_ref slate_bin_apps_fs="true">
            <file_name>rofs</file_name>
            <file_path>apps_proc/tools/fs/</file_path>
        </file_ref>
        <file_ref slate_bin_apps_fs="true">
            <file_name>pylib</file_name>
            <file_path>apps_proc/tools/</file_path>
        </file_ref>
        <file_ref slate_bin_common="true">
            <file_name>*</file_name>
<file_path>apps_proc/fw/app/common/partition_table_xml/</f
ile_path>
        </file_ref>
        <file_ref slate_bin_common="true">
            <file_name>*</file_name>
<file_path>apps_proc/fw/csp/storage/tools/partition_genera
tor/src/</file_path>
        </file_ref>

<!-- delete the below lines -->
        <file_ref fat_file="true" minimized="true">
            <file_name>flash.bin</file_name>
<file_path>common/build/bin/slate/binaries/</file_path>
        </file_ref>
        <file_ref fat_file="true" minimized="true">
            <file_name>partition_table.xml</file_name>
<file_path>common/build/bin/slate/binaries/</file_path>
        </file_ref>
        <file_ref fat_file="true" minimized="true">
            <file_name>partition_1.bin</file_name>
<file_path>common/build/bin/slate/binaries/</file_path>
        </file_ref>

<!-- delete the below lines -->
        <step filter="slate" type="exec">
            <exec_dir>./bin/slate/binaries/</exec_dir>
            <params>python @tool_name
partition_table_sw5100_wdp.xml
partition_table.xml</params>
            <tool_name>rename_file.py</tool_name>
        </step>
        <step filter="slate_qcc5100" type="exec">

```

```

        <exec_dir>bin/slate/binaries</exec_dir>
        <params>python @tool_name -b 0x10000 -f
partition_table.xml</params>
        <tool_name>generator.py</tool_name>
    </step>

```

- Navigate to APPS.SLATE.3.0/apps_proc/build/manifest.xml

- ```

<!-- Replace with the below content in
APPS.SLATE.3.0/apps_proc/build/manifest.xml --!>

```

```

<config>
 <image_tree>
 <name>APPS.SLATE.3.0</name>
 <build_id>APPS.SLATE.3.0-02245-
STD.NOFLASH_PSRAM.INT-2</build_id>
 <branch_point_revision>None</branch_point_revision>
 <revision>02245</revision>
 <spin>1</spin>
 <client>STD.NOFLASH_PSRAM.INT</client>
 </image_tree>
</config>

```

- Follow the below commands to build. Please note that NOFLASH\_PSRAM variant compiles out sensors for space on slate

| Build environment | Build command                                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linux             | <ul style="list-style-type: none"> <li>◦ Build images: ./build.sh --noflash_psram --enable_disp_host --enable_audio</li> <li>◦ Clean and rebuild: ./build.sh -c --noflash_psram --enable_disp_host --enable_audio</li> </ul> |
| Windows           | <ul style="list-style-type: none"> <li>◦ Build images: Build.bat --noflash_psram --enable_disp_host --enable_audio</li> <li>◦ Clean and rebuild: Build.bat -c --noflash_psram --enable_disp_host --enable_audio</li> </ul>   |

**Note 1:** In this release, persistency is enabled on NFWP variant as well. Persistent store is now stored in Atherton eMMC for this variant. By default, CONFIG\_ENABLE\_RMA flag is enabled on build\bsp\nonsecure\build\buildConfig.cfg. To disable persistency on this variant, remove the flag from build.cfg

**Note 2:** Since the PS Store on NFWP uses Glink, the PS Store will not be available till the corresponding Glink channels are available and initializations are done. This will have limitation on the usecases that needs to access the PS data during boot up or before the PS initialization is completed.

#### 4.2.7.2 Build Audio subsystem DSP image

Customers need to get “Xtensa\_2021.7 HiFi5\_slate\_v1\_1” from Cadence. Cadence is the third-party company who owns the xtensa tool.

1. Set XTENSA\_ROOT\_PATH environment variable before compilation (set the path where Xtensa tools are installed).

- For example, on Windows:

```
set PYTHON_ROOT=<path_of_python3.8.2>
```

```
set XTENSA_ROOT_PATH=<path_of_installed_tools_upto_XtDevTools>
```

```
set XTENSA_CORE=HiFi5_slate_v1_1
```

```
set XTENSA_SW_REL=RI-2021.7-win32
```

```
set XTENSA_HW_REL=RI-2021.7-win32
```

- For example, on Linux:

```
export PYTHON_ROOT=<path_of_python3.8.2>
```

```
export XTENSA_ROOT_PATH=<path_of_installed_tools_upto_XtDevTools>
```

```
export XTENSA_CORE=HiFi5_slate_v1_1
```

```
export XTENSA_SW_REL=RI-2021.7-linux
```

```
export XTENSA_HW_REL=RI-2021.7-linux
```

2. Open a command prompt and navigate to the following directory:

```
cd <target_root>\DSP.SLATE.1.1\audio_ss_proc\build\
```

3. Based on the build environment, run the appropriate command from the table:

#### For NOR Flash(MEP and WDP) DSP Build

| Build environment | Build command                                                                                   |
|-------------------|-------------------------------------------------------------------------------------------------|
| Linux             | ◦ Build images:<br>python ./build.py -c qcc5100 -o all -f USES_MEMANALYSIS,DYNL,GEN_SHARED_LIBS |
| Windows           | ◦ Build images:<br>python ./build.py -c qcc5100 -o all -f USES_MEMANALYSIS,DYNL,GEN_SHARED_LIBS |

For NO Flash PSRAM variant, edit the diag scons file to use SHMEM for logging.

Filename: <target\_root>\DSP.SLATE.1.1\audioss\_proc\core\services\diag\build\diag.scons

Change 1 : Disable PSRAM logging support on DSP (switch to SHMEM)

```
- if 'NOFLASH_NOPSRAM' in env['CCFLAGS']:
- env.Append(CPPDEFINES=['DIAG_SHMEM_SUPPORT'])
- else:
- env.Append(CPPDEFINES=['DIAG_PSRAM_SUPPORT'])
-
+
+ env.Append(CPPDEFINES=['DIAG_SHMEM_SUPPORT'])
+
```

Change 2 : Compile corresponding C file

```
- if 'NOFLASH_NOPSRAM' in env['CCFLAGS']:
- DIAG_SOURCES += [
- '${BUILDPATH}/DCM/common/src/diagcomm_io_shmem.c',
-]
- else:
- DIAG_SOURCES += [
- '${BUILDPATH}/DCM/common/src/diagcomm_io_psram.c',
+ DIAG_SOURCES += [
+ '${BUILDPATH}/DCM/common/src/diagcomm_io_shmem.c',
```

#### 4.2.8 Build WLAN

WLAN image is released as a binary and no build compilation is needed.

#### 4.2.9 Build BTFM

BTFM image is released as a binary and no build compilation is needed.

## 4.2.10 Build Video

Video image is released as a binary and no build compilation is needed.

## 4.3 Generate META build

For **APQ distro sw5100-law-2-0\_ap\_standard\_oem**, please use contents\_APQ.xml instead of default contents.xml (Path: <target\_root>/SW5100.LAW.2.0/contents\_APQ.xml).

For **AMSS distro sw5100-law-2-0\_amss\_standard\_oem**, please use default contents.xml (Path: <target\_root>/SW5100.LAW.2.0/contents.xml).

1. Before comping the META, please copy all HLOS images:

```
cd /LAW.VENDOR.14.0/LINUX/android/; mkdir -p out/target/product device/qcom/monaco-kernel
```

```
cp -rvf <HLOS_BUILD>/LINUX/out/target/product/monaco_go_aon
<target_root>/LAW.VENDOR.14.0/LINUX/android/out/target/product/monaco_go_aon
```

```
cp -rvf <HLOS_BUILD>/LINUX/android/device/qcom/monaco-kernel/vmlinux
<target_root>/LAW.VENDOR.14.0/LINUX/android/device/qcom/monaco-kernel/vmlinux
```

2. Go to the directory: cd <target\_root>/SW5100.LAW.2.0/common/build

3. Enter the command: (Python version to be used here is 3.8.2)

```
python build.py
```

**Notes:** If you encounter the following error when recompiling Meta, please excute 'rm -rf bin/slate/binaries/APPS/fs/' first. Then you can continue by executing 'python build.py' to compile.

```
<class 'FileExistsError':>: [Errno 17] File exists: 'bin/slate/binaries/APPS/fs/packfile'
```

## 4.4 Generate QCC5100 (NOR Flash) META build and Sideload

### 4.4.1 For NOR Flash

1. Build APPS.SLATE.3.0 and Audio DSP as described above
2. Go to the directory: cd <target\_root>/SW5100.LAW.2.0/common/build
3. Enter the below commands (Python version to be used here is >=3.8.2)

```
installs pyelftools if not installed in python 3.0
pip install pyelftools
```

```
creates the qcc5100.elf and flash.bin
```

```
python build.py --
```

```
wflow_filter=slate,slate_qcc5100,qcc5100_copy
```

```
Navigate to common\build\bin\slate\binaries and launch a
command shell
```

```
run the below adb commands
```

```
adb root
```

```
adb shell mount -o rw,remount /vendor/firmware_mnt
```

```
adb push partition_table.xml /vendor/firmware_mnt/image
```

```
adb push flash.bin /vendor/firmware_mnt/image
```

```
adb push qcc5100_ver_info.txt /vendor/firmware_mnt/image
```

```
adb push partition_1.bin /vendor/firmware_mnt/image
```

4. Reboot the device after pushing the above files

#### 4.4.2 For NO\_FLASH\_PSRAM

1. Build APPS.SLATE.3.0 and Audio DSP as described above
2. Go to the directory: cd <target\_root>/SW5100.LAW.2.0/common/build
3. Enter the below commands (Python version to be used here is >=3.8.2)

4.

```
installs pyelftools if not installed in python 3.0
pip install pyelftools
```

```
creates the qcc5100.elf
```

```
python build.py --
```

```
wflow_filter=slate,slate_qcc5100,qcc5100_copy
```

```
split qcc5100.elf
```

```
cd common\build\bin\slate\binaries
```

```
python2.7 is required to run pil-splitter.py
```

```
python <meta_root>\common\config\pil-splitter.py qcc5100.elf
slatefw
```

```
create BIN directory and move all slatefw*. files to BIN
mkdir BIN
```

```
mv slatefw* BIN
cd BIN
```

```
run the below adb commands
adb root
```

```
adb shell mount -o rw,remount /vendor/firmware_mnt
```

```
adb wait-for-device push . /vendor/firmware_mnt/image
```

?????

5.

```
cd common\build\bin\slate\binaries\BTFW\
rename QCC5100_rampatch_mbnv7.elf as slbtfw20.mbn
adb shell umount /vendor/bt_firmware
adb shell mount -t vfat -o rw /dev/block/bootdevice/by-
name/bluetooth /vendor/bt_firmware
adb shell mount -o rw,remount /vendor/bt_firmware
adb push slnv20.bin /vendor/bt_firmware/image/
adb push slbtfw20.mbn /vendor/bt_firmware/image/
adb shell sync
```

6. Reboot the device after pushing the above files

## 4.5 Program CDT

For procedures such as flashing images, CDT configuration, refer to *SW5100 Android Wearables Software Programming Guide* (80-20387-200)

## 5. Memory configuration

### Memory configuration

### Supported ASICs

*This software can be used with the MSM™ ASICs and revisions, with the indicated release quality. ASIC revisions available at the time of this release are assumed to be supported unless otherwise indicated.*

| Chip Name | Variant (PRR-BB) | Feature_ID | Source Configuration (s) | Note                                                                  | Sample Type |
|-----------|------------------|------------|--------------------------|-----------------------------------------------------------------------|-------------|
| SW5100-0  | 001-AA           | 0x1        | 0                        | SW5100 (001-0-AA), MPSP688, 4 x A53, MEP configuration with modem     | ES          |
| SW5100P-0 | 001-AA           | 0x1        | 0                        | SW5100P (001-0-AA), MPSP688, 4 x A53, MEP configuration with no modem | ES          |

### ASICs no longer supported

| ASIC Hardware | Hardware Version | Variant (PRR-BB) | Feature_ID | Source Configuration (s) | Note                                                                  | Sample Type |
|---------------|------------------|------------------|------------|--------------------------|-----------------------------------------------------------------------|-------------|
| SW5100-0      | v1.0             | 000-AA           | 0x1        | 0                        | SW5100 (000-0-AA), MPSP688, 4 x A53, MEP configuration with modem     | EA, ES      |
| SW5100P-0     | v1.0             | 000-AA           | 0x1        | 0                        | SW5100P (000-0-AA), MPSP688, 4 x A53, MEP configuration with no modem | EA, ES      |

### Platform Information

The following platforms are supported in this release:

|     |                                                                                                                                                                   |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IDP | IDP, SW5100 MEP V1.0, SS 2GB LP4X+32GB EMMC, PMW5100, PM8010, WTR3925                                                                                             |
| WDP | DTP, WDP5100, DVT1.1, SW5100 MEP EXT SECURE, KG EPOP, 2GB LPDDR4X+32GB EMMC, 32MB FLASH, 16MB PSRAM, PMW5100, PM8010, QCC5100, WTR3925, QFM5518, QPA5590, QET6106 |

## Memory Platform Information

The following memory platform is supported in this release

- Package-on-package (PoP) high-speed eMCP memory
  - 2GB LPDDR4X SDRAM, 32GB eMMC 5.1
  - DDR Frequencies supported

|         |                   |
|---------|-------------------|
| Low SVS | 200 MHz           |
| SVS     | 768 MHz, 1017 MHz |
| NOM     | 1807 MHz          |
| Turbo   | 2092 Mhz          |

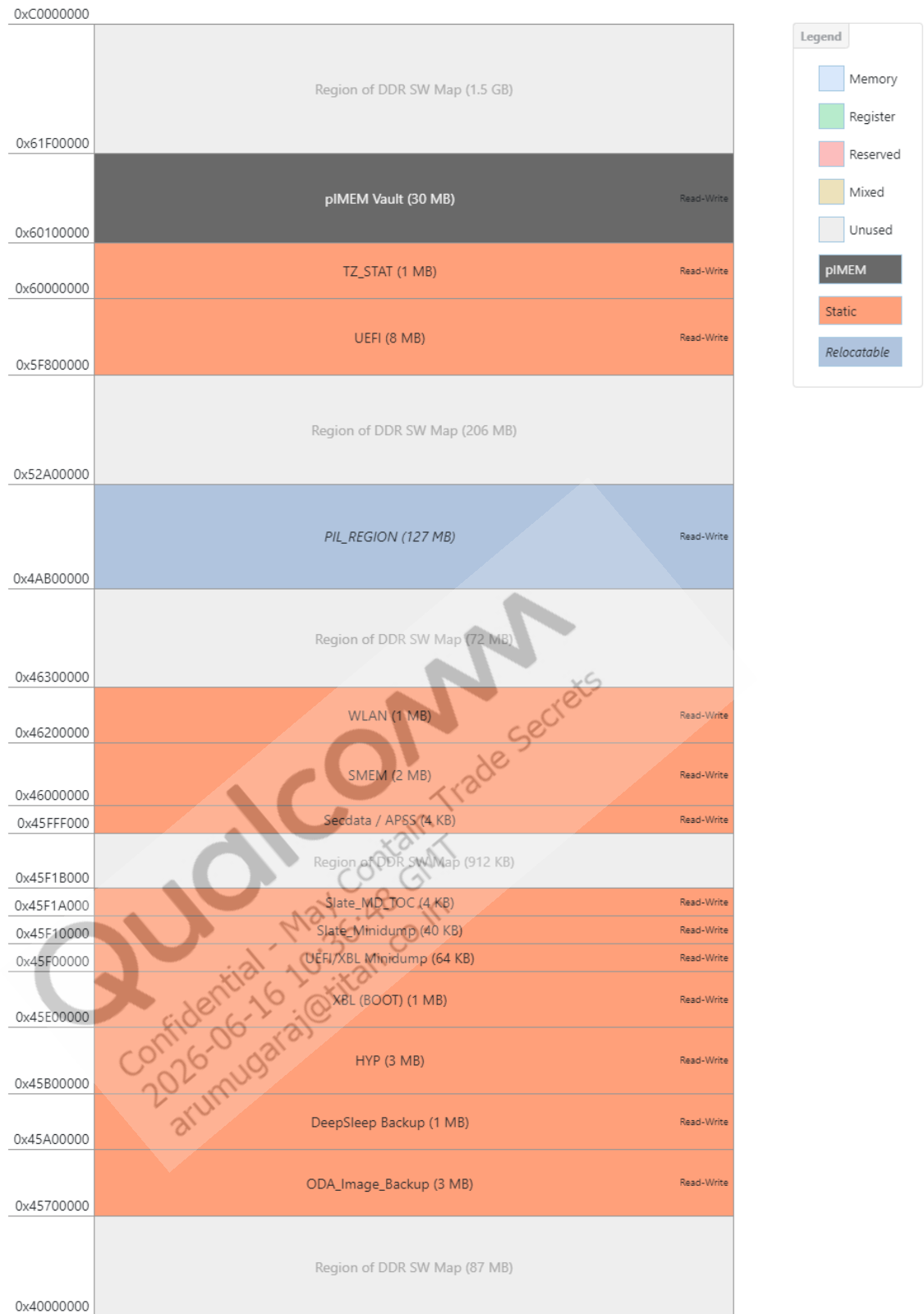
## RF Card

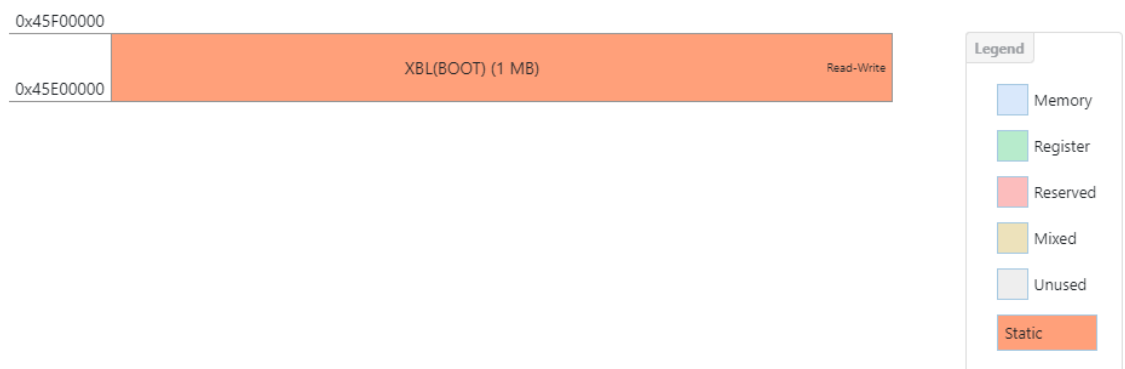
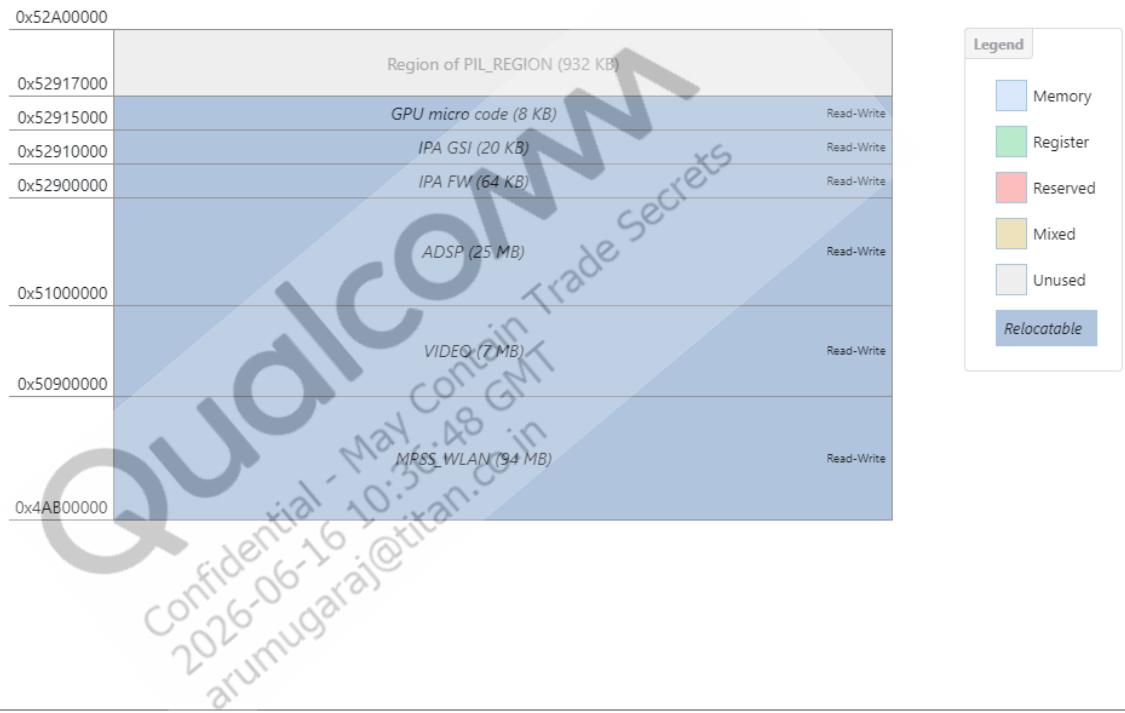
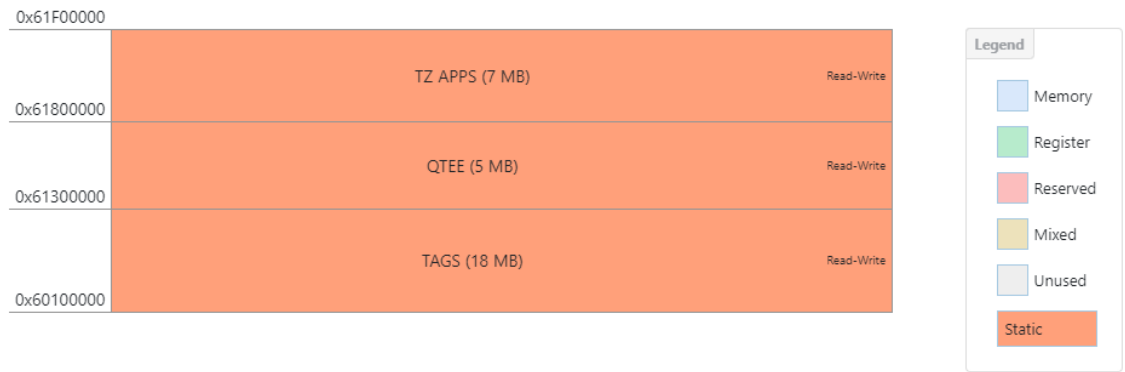
The following memory platform is supported in this release

|                                                                  |
|------------------------------------------------------------------|
| WTR3925_ROW                                                      |
| WTR3925 + QPA5590 + QET6105 + QFM5518 + QAT5515 + QDM2310 Non-CA |

## Memory Map Info (MSM):

**NOTE:** This is a preliminary memory map and may change in the future releases. It is for reference-only in this release and should not be used for optimization.





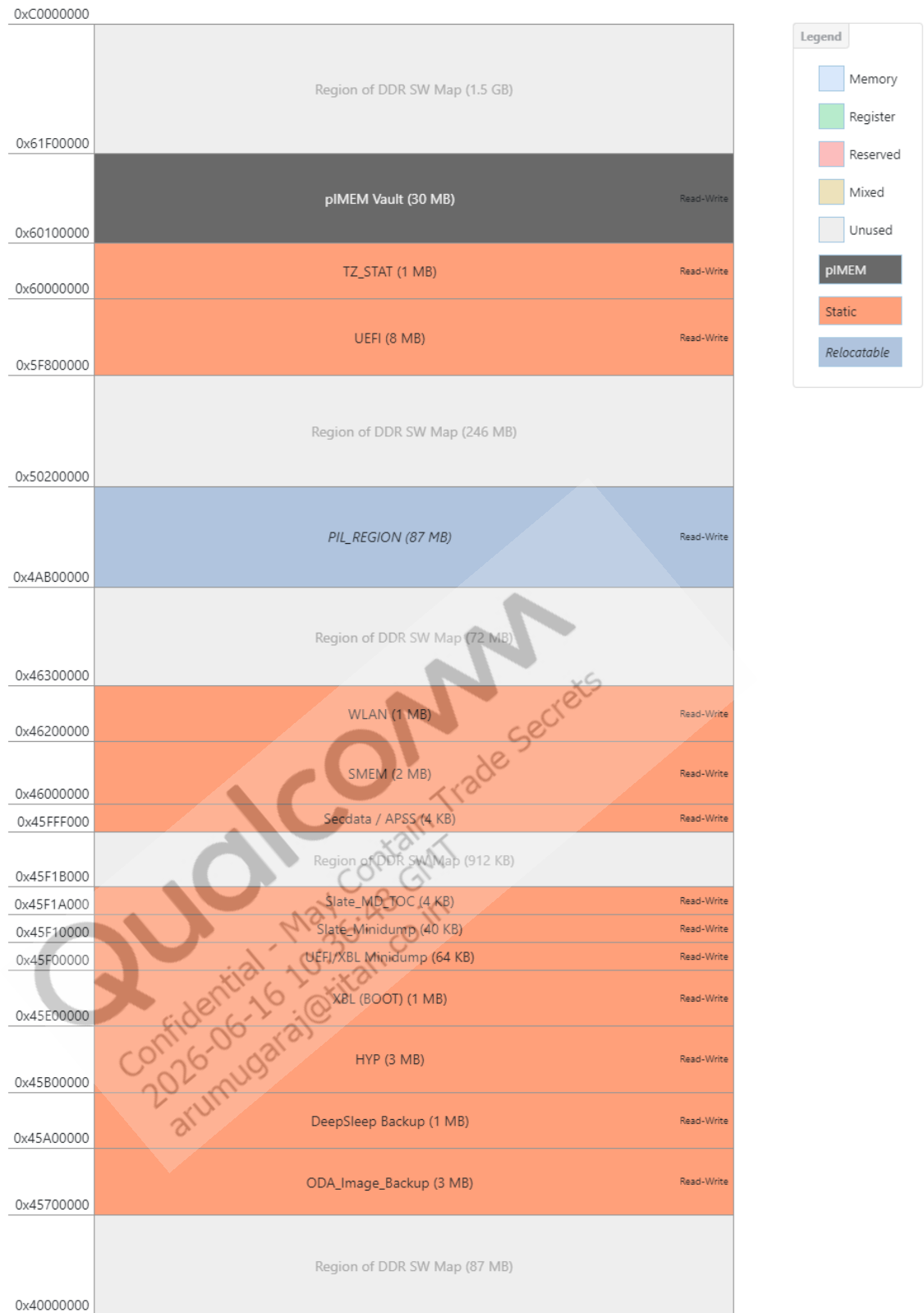
|            |                                     |            |
|------------|-------------------------------------|------------|
| 0x45B00000 |                                     |            |
| 0x45AA4C00 | Region of DeepSleep Backup (365 KB) |            |
| 0x45A94C00 | TZ_Config_Backup (64 KB)            | Read-Write |
| 0x45A92C00 | DDR (8 KB)                          | Read-Write |
| 0x45A80000 | QUP FW (75 KB)                      | Read-Write |
| 0x45A00000 | RPM Backup (512 KB)                 | Read-Write |

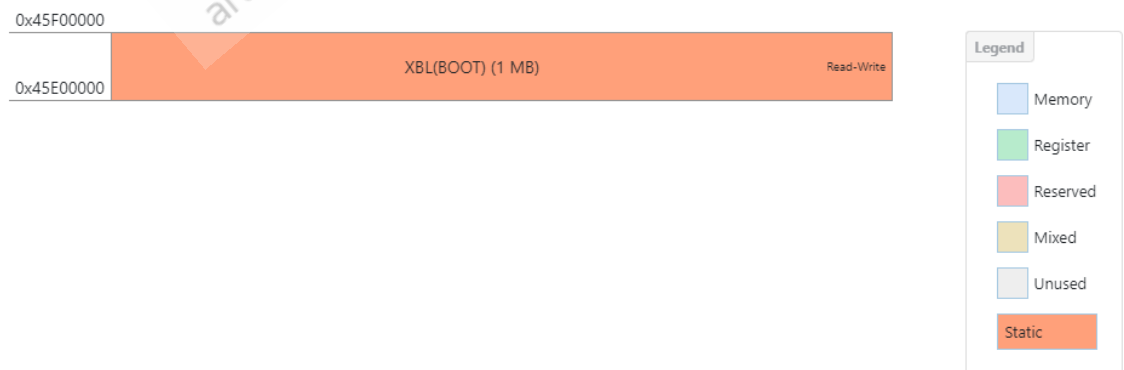
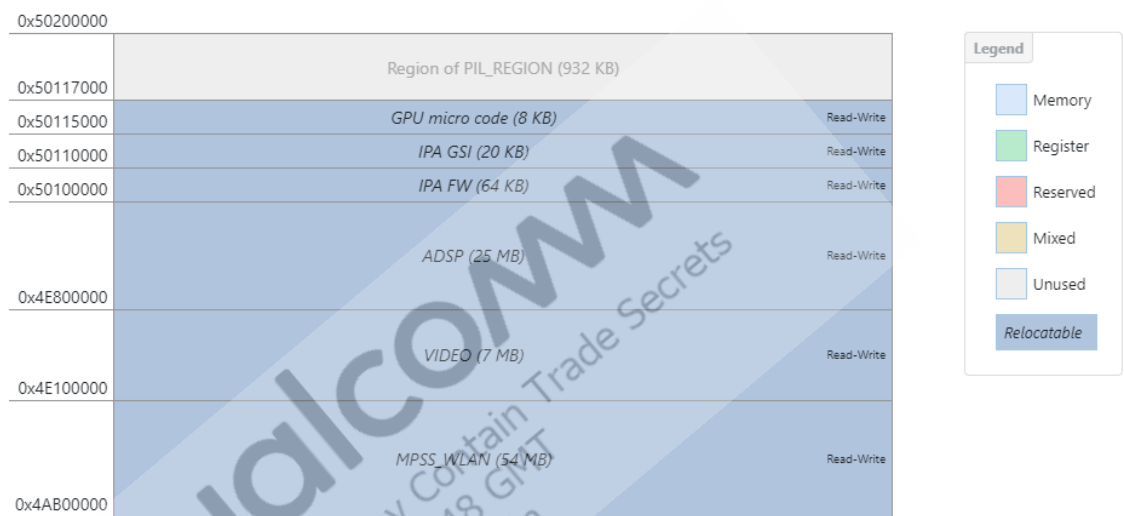
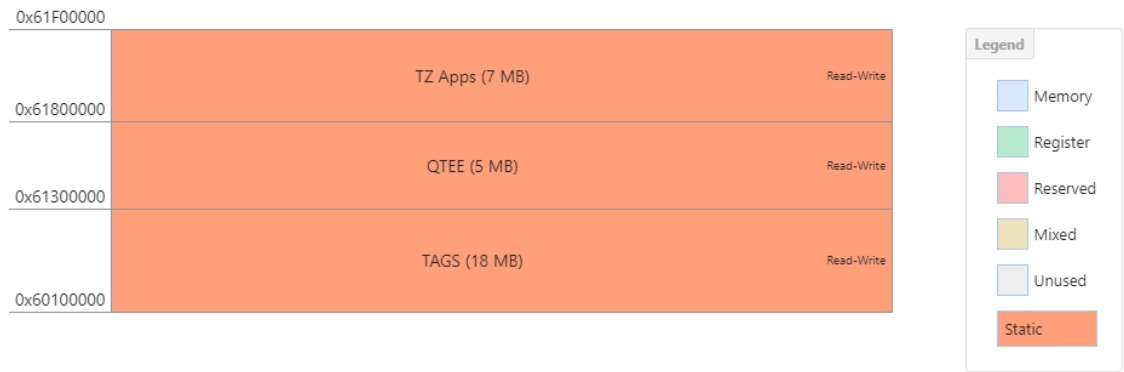
**Legend**

- Memory
- Register
- Reserved
- Mixed
- Unused
- Static

Memory Map Info (APQ):

**NOTE:** This is a preliminary memory map and may change in the future releases. It is for reference-only in this release and should not be used for optimization.





|            |                                     |            |
|------------|-------------------------------------|------------|
| 0x45B00000 | Region of DeepSleep Backup (365 KB) |            |
| 0x45AA4C00 |                                     |            |
| 0x45A94C00 | TZ_Config_Backup (64 KB)            | Read-Write |
| 0x45A92C00 | DDR Training data (8 KB)            | Read-Write |
| 0x45A80000 | QUP FW (75 KB)                      | Read-Write |
| 0x45A00000 | RPM Backup (512 KB)                 | Read-Write |

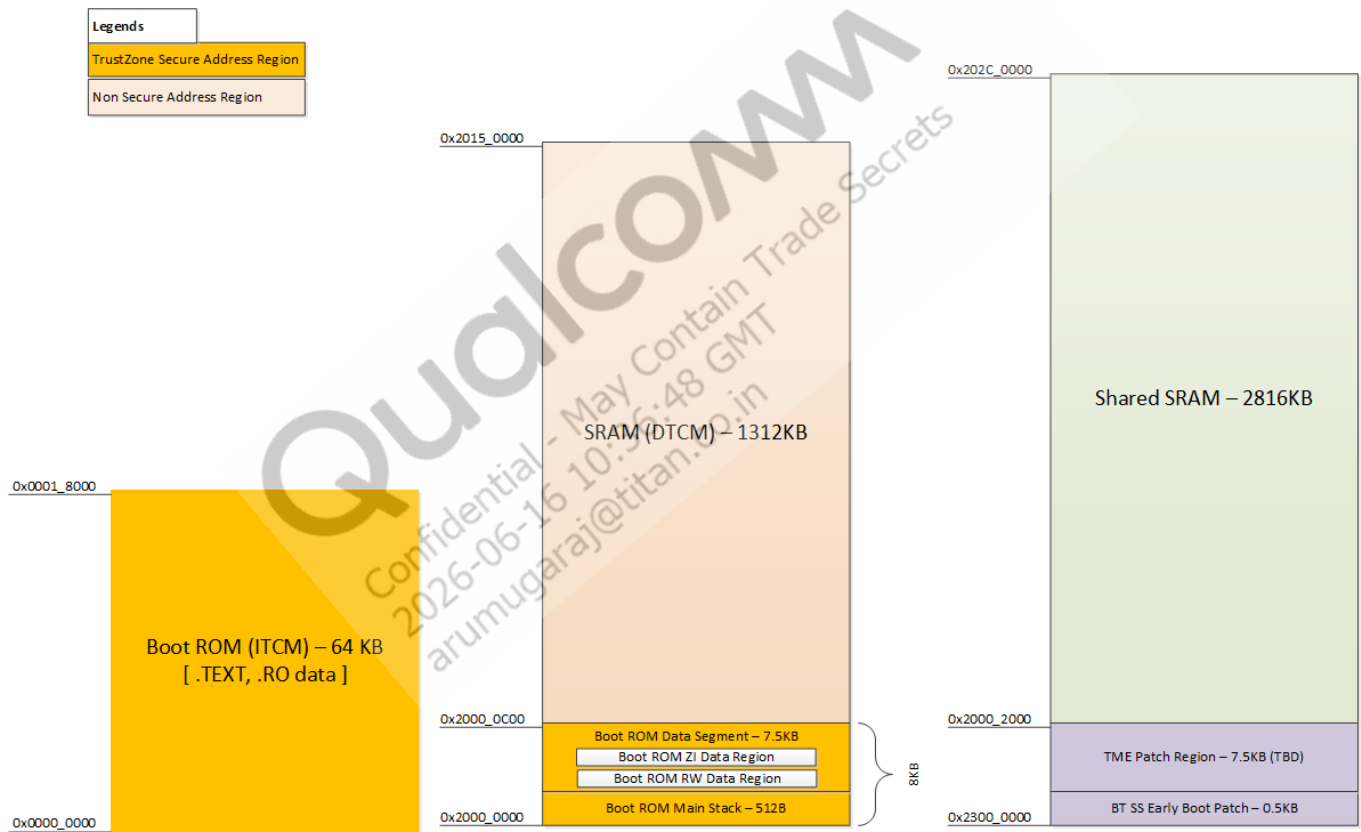
**Legend**

- Memory
- Register
- Reserved
- Mixed
- Unused
- Static

Memory Map Info (QCC5100):

**NOTE:** This is a preliminary memory map and may change in the future releases. It is for reference-only in this release and should not be used for optimization

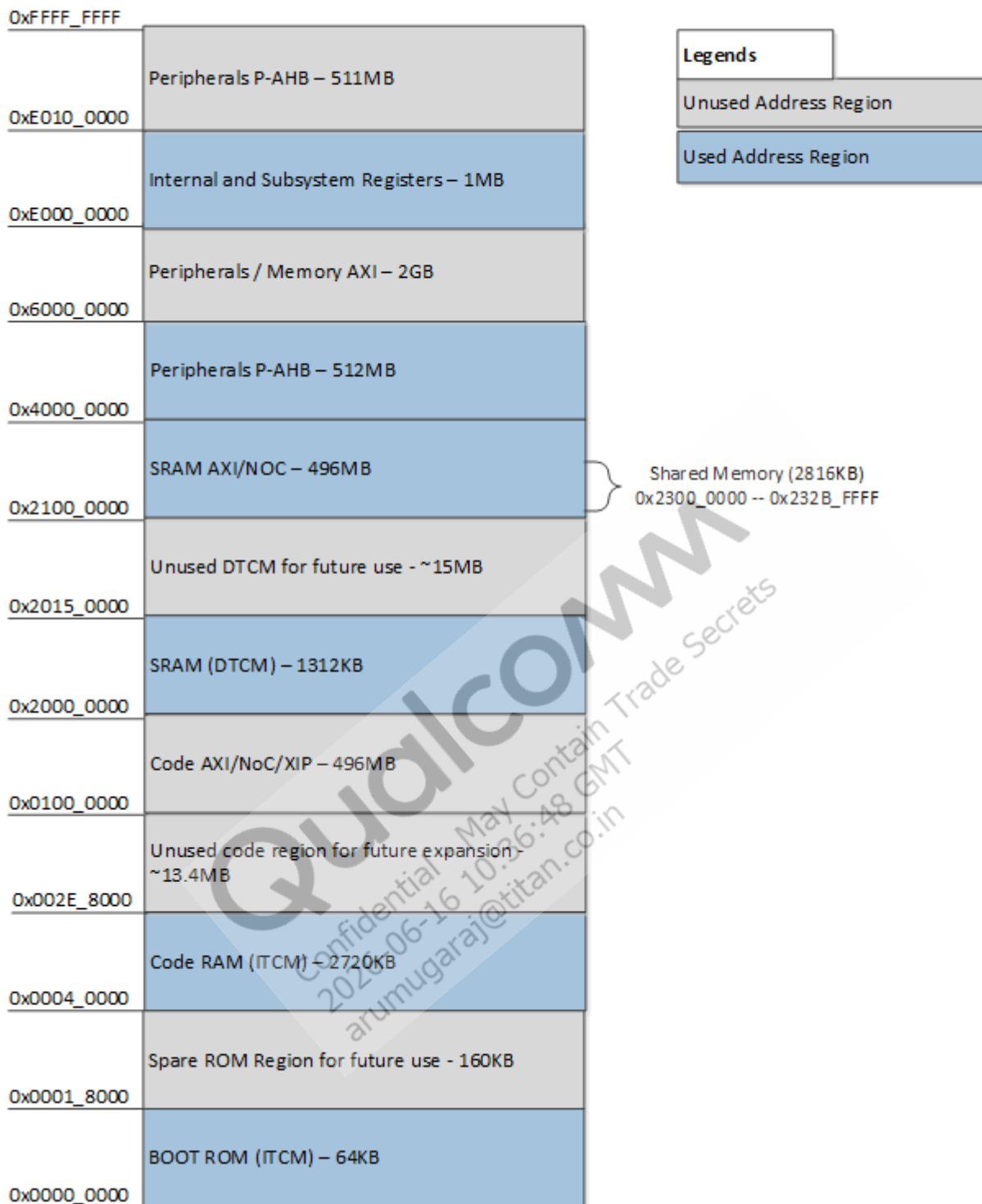
The following figure captures the logical memory map at time of boot.



The following figure captures the logical view of content residing in various memories on QCC5100 in mission mode as view by APPS/M55 subsystem.

The M55 memory map is below

# M55 Memory Map



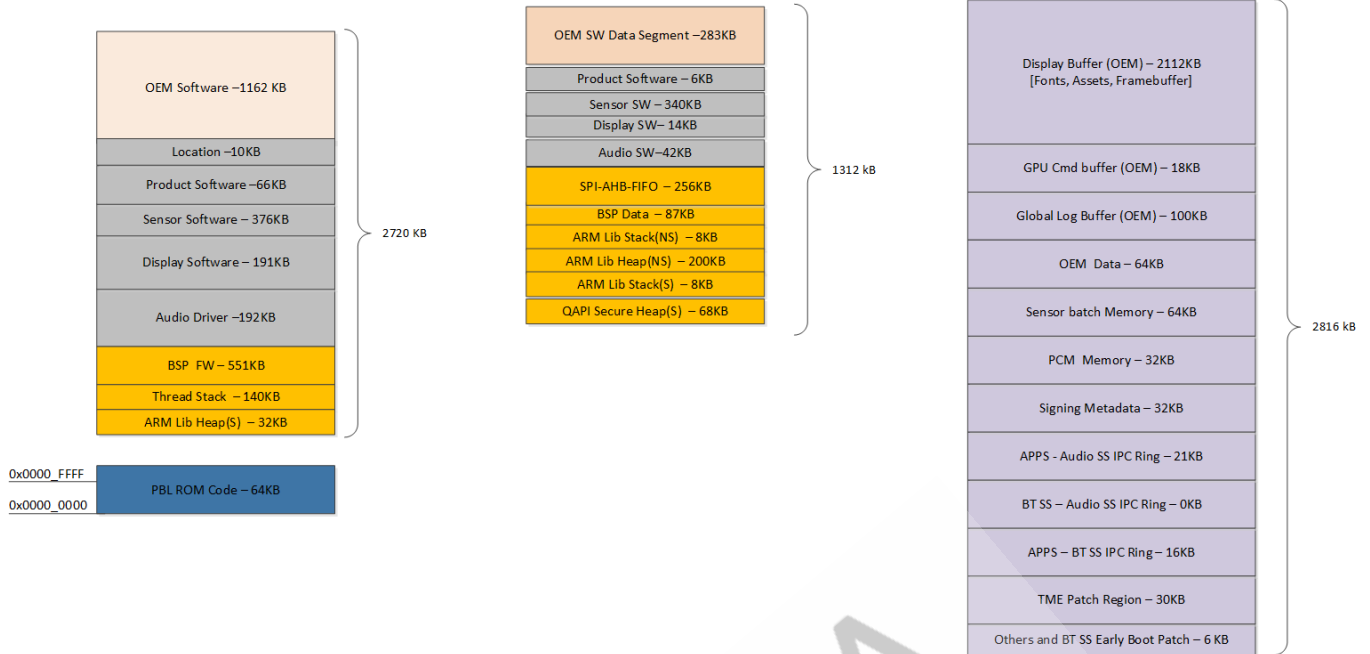
Further, Slate supports three skews, the memory map of mission mode image of Slate APSS is as follows

a. NOFLASH\_NOPSRAM Variant

## ITCM(Code)

## DTCM(Data)

## Shared SRAM



## b. SPINOR+PSRAM Variant

### NOR Flash - Snapshot

|                               |
|-------------------------------|
| QCC_RAM_patch_IMAGE(BT Patch) |
| dm20_IMAGE (NVM)              |
| NS_XIP_IMG (RAM Image)        |
| RamDump(55K)                  |
| BT Persistent Store(55K)      |
| BT Storage (131K)             |
| RA1, RA2 Empty                |
| Flash XIP Demo Code           |

### ITCM(Code)

|                               |
|-------------------------------|
| OEM Software –1044 KB         |
| Product Software –135KB       |
| Sensor Software – 215KB       |
| BT Software-Location – 262 KB |
| Display Software – 140KB      |
| Audio Driver –160KB           |
| BT Stack – 140KB              |
| BSP FW – 551KB                |
| ARM Lib Heap(NS) – 32KB       |

2720 KB

### PSRAM (XIP+Data)

|                         |
|-------------------------|
| Demo Code-286B          |
| Patch SW – 707B         |
| Audio SW – 42KB – 212K  |
| Sensor SW – 340KB       |
| Display SW – 14KB       |
| Sensors XIP – 171K      |
| Audio DIAG data – 102KB |
| OEM Space               |

### NOR Flash XIP

|                     |
|---------------------|
| Flash XIP Demo Code |
|---------------------|

### DTCM(Data)

|                            |
|----------------------------|
| OEM SW Data Segment –193KB |
| Product Software – 54KB    |
| Sensor SW – 340KB          |
| Display SW – 14KB          |
| Audio SW–35KB              |
| BT Stack – 140KB           |
| SPI-AHB-FIFO – 256KB       |
| BSP Data – 87KB            |
| ARM Lib Stack(NS) – 8KB    |
| ARM Lib Heap(NS) – 200KB   |
| ARM Lib Stack(S) – 8KB     |
| IPC Heap – 24KB            |
| QAPI Secure Heap(S) – 68KB |

1312 KB

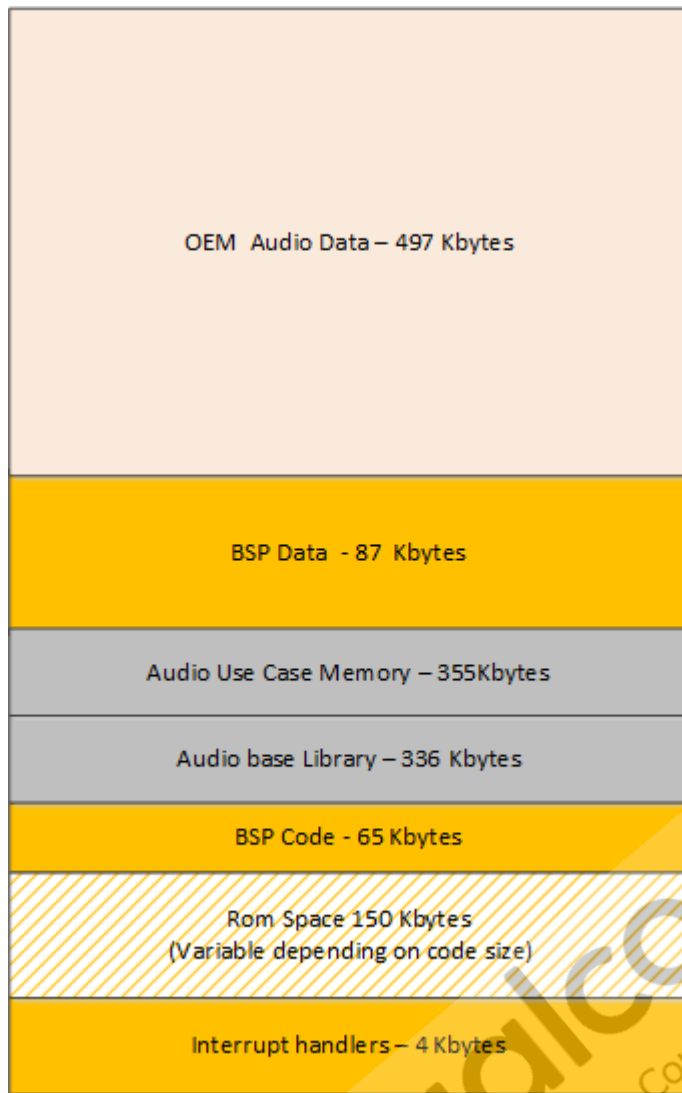
### Shared SRAM

|                                                               |
|---------------------------------------------------------------|
| Display Buffer (OEM) – 2112KB<br>[Fonts, Assets, Framebuffer] |
| GPU Cmd buffer (OEM) – 18KB                                   |
| Global Log Buffer (OEM) – 100KB                               |
| OEM Data – 64KB                                               |
| Sensor batch Memory – 64KB                                    |
| PCM Memory – 32KB                                             |
| Signing Metadata – 32KB                                       |
| APPS - Audio SS IPC Ring – 21KB                               |
| BT SS – Audio SS IPC Ring – 0KB                               |
| APPS – BT SS IPC Ring – 16KB                                  |
| TME Patch Region – 30KB                                       |
| Others and BT SS Early Boot Patch – 6 KB                      |

2816 KB

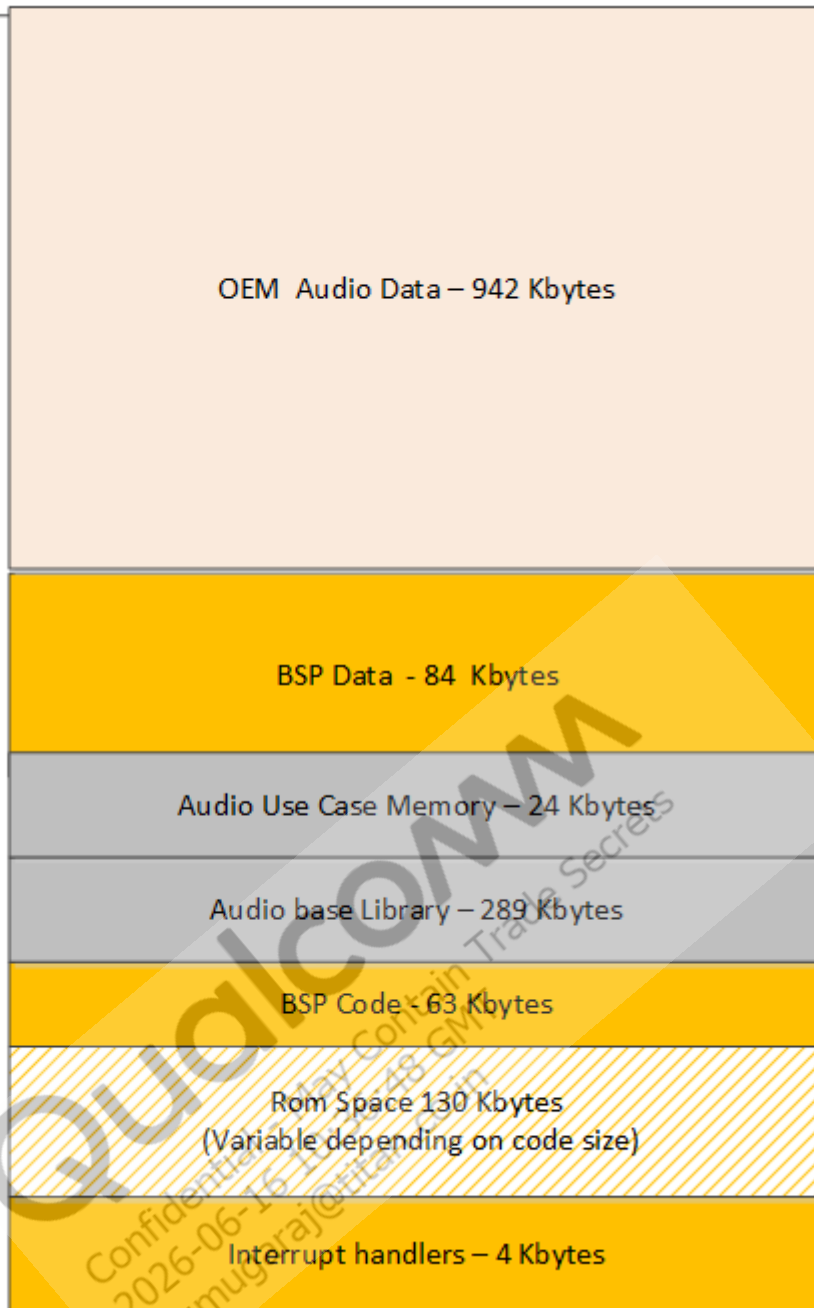
Similarly, the logical view of AudioSS memory for different skews is as below

## a. SPINOR + PSRAM Variant



b. NOFLASH\_NOPSRAM Variant

0x2027\_FFFF



0x2010\_0000

## 6. New Features

In this release overall software quality are meeting CS metrics

UX offload continue to improve in this release, if you need to enable UX offload, follow the instruction below:

- UX offload(not enabled by default in release build), please follow below guidance if you need to practice UX offload feature
  - NON-HLOS enablement instruction
    - rebuild QCC5100 APSS application processor image with UXOffload configs, more details refer to **Build Instructions** section.
  - HLOS enable instructions
    - In QWSI, set UxOffload flags in make file:device/qcom/qssi\_32go/qssi\_32go.mk then rebuild QWSI image.
      - TARGET\_SUPPORTS\_UX\_OFFLOAD := true
      - TARGET\_SUPPORTS\_AIDL\_UI\_OFFLOAD:= true
    - In VENDOR, set UxOffload flags in make file:device/qcom/monaco\_go\_aon/monaco\_go\_aon.mk then rebuild VENDOR image.
      - TARGET\_SUPPORTS\_UI\_OFFLOAD := true --> TARGET\_SUPPORTS\_UI\_OFFLOAD := false
      - TARGET\_SUPPORTS\_AIDL\_UI\_OFFLOAD := true
    - generate super image and flash apps.

## 7. Limitations

This chapter lists the issues and limitations reported for this release.

- New – Newly reported issues and limitations
- Unresolved – Previously reported issues and limitations
- Resolved – Previously reported issues and limitations that have been resolved and are no longer relevant

### Issues

Note: Following are the issues information for this release.

#### New Issues

#### Unresolved Issues

- CR/3935924 [NFWP] DUT fails to enable the Bluetooth after BLE connect and Pair followed by multiple reboots, randomly
- CR/4191491 [WDP][Display] In Ambient mode BT symbol is NOT disappearing when BT SSR triggered with minidump enabled
- CR/3985980 [LAW2.0] [IDP NOR] [ACTIVE][Multi-GATT]: Unable to perform Bulk data transfer from second client in Multi LECOC client scenario

#### Resolved

- CR/4191437 [IDP][WDP][Display] Display got stuck and NOT resuming post slate SSR with minidump enabled

### Limitation

- For the NOFLASH\_PSRAM variant, the boot time is close to 7 minutes. This is due to sensors being compiled out.

## 8. Test Reports

### IDP Nor Variant

SW: IDP Nor variant build

HW: IDP device with Nor Flash and PSRAM support

Tech Area	Sub Area	Test Case Name	Test Result
Platform	Boot	EDL Download	Pass
		Fastboot Download	Pass
		Slate boot from flash with 1st boot	Pass
		Slate boot from flash after reboot	Pass
		Meta Info Check	Pass
		Android version Check	Pass
		Kernel version Check	Pass
		adb reboot	Pass
		Fastboot	Pass
		Alpaca Control	Pass
		dmesg	Pass
		logcat	Pass
	Peripherals	UART	Pass
		Slate UART	Pass
	Subsystem	Sub-system online/offline	Pass
		SSR test on ADSP	Pass
		SSR test on MODEM(WCNSS)	Pass
	Dump	Atherton RAM Dump	Pass
		Slate RAM Dump	Pass
	Baseport	CPU test	Pass
		GPU test	Pass
		DDR test	Pass
	Dm-Verity	Enable Verity	Pass
		Disable verity	Pass
Display/Touch	Display	DI_Brightness	Pass
	Display	DI_ContinuousSplashScreen	Pass
	Touch	TP_check_after_reboot	Pass
Camera	Front_Camera	Camera Preview srMax	Pass
	Front_Camera	Camera ZSL Snapshot 5MP	Pass
	Front_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Front_Camera	Camcorder Recording@1080p/30fps	Pass
	Back_Camera	Camera Preview srMax	Pass
	Back_Camera	Camera ZSL Snapshot 16MP	Pass
	Back_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Back_Camera	Camcorder Recording @1080p/30fps	Pass
Audio	Audio Encode	AE_amrmb_3gp_8k_mono	Pass
	Audio Decode	AD_NGTC-AACLC-mono-11025Hz-64000bps_aac	Pass
	Audio Decode	AD_NGTC-48kHz-mono_mp3	Pass
	Audio Decode	AD_NGTC-PCM-16bit-stereo_wav	Pass
	Audio Decode	AD_Audio_playback_volume verification	Pass

Video	Video Decode	H264 Video playback_1080p	Pass
	Video Decode	HEVC Video playback_1080p	Pass
	Video Decode	VP9 Video playback_1080p	Pass
UI	Animation	Power on android animation	Pass
	Launcher	Lock Screen	Pass
		Unlock to Home Screen	Pass
		Scroll in Home Screen	Pass
	Notification Bar	BT ON/OFF	Pass
		Brightness	Pass
	Settings	Launching of Settings app	Pass
		Power off	Pass
		Restart	Pass
		Factory reset	Pass
Offload	Power Manager	Power key to Wake up	Pass
		Touch to Wake up	Pass
	Tracker	Tracker mode enter and exit	Pass
	TWM	TWM enter and exit	Pass
Sensor	AON ON	AON on under Trancker mode	Pass
	AON OFF	AON off under Trancker mode	Pass
	Sensor	Accel	Pass
	Sensor	Gyro	Pass
	Sensor	Mag	Pass
	Sensor	ALS/Light	Pass
Connectivity	Co-work	WLAN+BT enable at same time	Pass
		wlan off/on 5 times	Pass
		WPA2+AES network connction	Pass
		Ping 1min	Pass
		AP Remenber after off/on	Pass
	BT	Enable/Disable BT 5 times	Pass
		Inquiry devices	Pass
		Pair devices from DUT	Pass
		Unpair devices from DUT	Pass
		DeviceSwitch_Playback_speaker_BT	Pass
		DeviceSwitch_Voice_Call_speaker_BT	Pass
GPS	GPS	GNSS_Position_Fix	Pass
Modem	IMS register/unregister	Register/Unregister IMS	Pass
	CU-LTE	LTE-CU MT/MO Call	Pass
	CU-LTE	LTE-CU Data	Pass
	CT-LTE	LTE-CT MT/MO Call	Pass
	CT-LTE	LTE-CT Data	Pass
	CMCC LTE	LTE-CMCC MT/MO Call	Pass
	CMCC LTE	LTE-CMCC MT/MO SMS/MMS	Pass
	CMCC LTE	LTE-CMCC Data	Pass
	VOLTE	VoLTE MT/MO Call	Pass
	Adversial-APM	Adversial-APM_ON-OFF	Pass

## WDP Nor Variant

SW: WDP Nor variant build

HW: WDP device with Nor Flash and PSRAM support

Tech Area	Sub Area	Test Case Name	Test Result
Platform	Boot	EDL Download	Pass
		Fastboot Download	Pass
		Slate boot from flash with 1st boot	Pass
		Slate boot from flash after reboot	Pass
		Meta Info Check	Pass
		adb reboot	Pass
		Fastboot	Pass
	Peripherals	UART	Pass
		Slate UART	Pass
	Subsystem	Sub-system online/offline	Pass
		SSR test on ADSP	Pass
		SSR test on MODEM(WCNSS)	Pass
	Dump	Atherton RAM Dump	Pass
		Slate RAM Dump	Pass
	Baseport	CPU test	Pass
		GPU test	Pass
		DDR test	Pass
Display/Touch	Display	DI_Brightness	Pass
	Display	DI_ContinuousSplashScreen	Pass
	Touch	TP_check_after_reboot	Pass
Camera	Front_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Front_Camera	Camcorder Recording@1080p/30fps	Pass
	Back_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Back_Camera	Camcorder Recording @1080p/30fps	Pass
Audio	Audio Encode	AE_amrnb_3gp_8k_mono	Pass
	Audio Decode	AD_NGTC-AACLc-mono-11025Hz-64000bps_aac	Pass
Video	Video Decode	H264 Video playback_1080p	Pass
	Video Decode	VP9 Video playback_1080p	Pass
UI	Animation	Power on android animation	Pass
	Launcher	Lock Screen	Pass
		Scroll in Home Screen	Pass
	Notification Bar	BT ON/OFF	Pass
		Brightness	Pass
Offload	Tracker	Tracker mode enter and exit	Pass
	TWM	TWM enter and exit	Pass
	AON ON	AON on under Trancker mode	Pass
	AON OFF	AON off under Trancker mode	Pass
Sensor	Sensor	Accel	Pass
	Sensor	Gyro	Pass
	Sensor	Mag	Pass
Connectivity	Co-work	WLAN+BT enable at same time	Pass
	WLAN	wlan off/on 5 times	Pass
	BT	Enable/Disable BT 5 times	Pass
		Inquiry devices	Pass
		Pair devices from DUT	Pass
GPS	GPS	GNSS_Position_Fix	Pass
Modem	IMS register/unregister	Register/Unregister IMS	Pass
	CMCC LTE	LTE-CMCC MT/MO Call	Pass
	CMCC LTE	LTE-CMCC MT/MO SMS/MMS	Pass
	CMCC LTE	LTE-CMCC Data	Pass
	Adversial-APM	Adversial-APM_ON-OFF	Pass

## NOFLASH\_PSRAM Variant

SW: NOFLASH\_PSRAM variant build

HW: IDP NFNP device with no Flash and no PSRAM support

Scenario	Result
BT turn on and off	Pass
BT and BLE scan and pair	Pass
Pair and play music with BT headset	Pass
Outgoing HFP call from watch	Pass
View contacts/call history from watch paired to phone	Pass

Qualcomm  
Confidential - May Contain Trade Secrets  
2026-06-16 10:36:48 GMT  
arumugaraj@titan.co.in

Tech Area	Sub Area	Test Case Name	Test Result
Platform	Boot	EDL Download	Pass
		adb remount	Pass
		Load QCN	Pass
		Meta Info Check	Pass
		adb reboot	Pass
		Fastboot	Pass
	Peripherals	UART	Pass
		Slate UART	Pass
	Subsystem	Sub-system online/offline	Pass
	Dump	Atherton RAM Dump	Pass
		Slate RAM Dump	Pass
Display/Touch	Display	DI_Brightness	Pass
	Touch	TP_check_after_reboot	Pass
Camera	Front_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Front_Camera	Camcorder Recording@1080p/30fps	Pass
	Back_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Back_Camera	Camcorder Recording @1080p/30fps	Pass
Audio	Audio Encode	AE_amrnb_3gp_8k_mono	Pass
	Audio Decode	AD_NGTC-AACLC-mono-11025Hz-64000bps_aac	Pass
Video	Video Decode	H264 Video playback_1080p	Pass
	Video Decode	VP9 Video playback_1080p	Pass
Offload	Tracker	Tracker mode enter and exit	Pass
	TWM	TWM enter and exit	Pass
Connectivity	Co-work	WLAN+BT enable at same time	Pass
	WLAN	wlan off/on 5 times	Pass
	BT	Enable/Disable BT 5 times	Pass
		Pair devices from DUT	Pass
		Playback over BT Headset	Pass
		Pause/Start/Next/Last over BT Headset	Pass
		Voice Call over BT Headset	Pass
GPS	GPS	GNSS_Position_Fix	Pass
Modem	IMS register/unregister	Register/Unregister IMS	Pass
	CMCC LTE	LTE-CMCC MT/MO Call	Pass
	CMCC LTE	LTE-CMCC MT/MO SMS/MMS	Pass
	CMCC LTE	LTE-CMCC Data	Pass
	Adversial-APM	Adversial-APM_ON-OFF	Pass
Power	Suspend/Vddmin	Suspend/Vddmin check with SIM card and APM OFF	Pass
	Suspend/Vddmin	Suspend/Vddmin check with SIM card and APM ON	Pass

## APQ Variant

SW: IDP APQ Nor variant build

HW: IDP APQ device with Nor Flash and PSRAM support

Tech Area	Sub Area	Test Case Name	Test Result
-----------	----------	----------------	-------------

Platform	Boot	EDL Download	Pass
		Fastboot Download	Pass
		Slate boot from flash with 1st boot	Pass
		Slate boot from flash after reboot	Pass
		Meta Info Check	Pass
		Android version Check	Pass
		Kernel version Check	Pass
		adb reboot	Pass
		Fastboot	Pass
		Alpaca Control	Pass
		dmesg	Pass
		logcat	Pass
	Peripherals	UART	Pass
		Slate UART	Pass
	Subsystem	Sub-system online/offline	Pass
		SSR test on ADSP	Pass
		SSR test on MODEM(WCNSS)	Pass
	Dump	Atherton RAM Dump	Pass
		Slate RAM Dump	Pass
	Baseport	CPU test	Pass
		GPU test	Pass
		DDR test	Pass
	Dm-Verity	Enable Verity	Pass
		Disable verity	Pass
Display/Touch	Display	DI_Brightness	Pass
	Display	DI_ContinuousSplashScreen	Pass
	Touch	TP_check_after_reboot	Pass
Camera	Front_Camera	Camera Preview srMax	Pass
	Front_Camera	Camera ZSL Snapshot 5MP	Pass
	Front_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Front_Camera	Camcorder Recording@1080p/30fps	Pass
	Back_Camera	Camera Preview srMax	Pass
	Back_Camera	Camera ZSL Snapshot 16MP	Pass
	Back_Camera	Camcorder live snapshot_Video_h264_Max@1080p	Pass
	Back_Camera	Camcorder Recording @1080p/30fps	Pass
Audio	Audio Encode	AE_amrnb_3gp_8k_mono	Pass
	Audio Decode	AD_NGTC-AACLC-mono-11025Hz-64000bps_aac	Pass
	Audio Decode	AD_NGTC-48kHz-mono_mp3	Pass
	Audio Decode	AD_NGTC-PCM-16bit-stereo_wav	Pass
	Audio Decode	AD_Audio_playback_volume verification	Pass
Video	Video Decode	H264 Video playback_1080p	Pass
	Video Decode	HEVC Video playback_1080p	Pass
	Video Decode	VP9 Video playback_1080p	Pass

UI	Animation	Power on android animation	Pass
	Launcher	Lock Screen	Pass
		Unlock to Home Screen	Pass
		Scroll in Home Screen	Pass
	Notification Bar	BT ON/OFF	Pass
		Brightness	Pass
	Settings	Launching of Settings app	Pass
		Power off	Pass
		Restart	Pass
		Factory reset	Pass
Offload	Power Manager	Power key to Wake up	Pass
		Touch to Wake up	Pass
	Tracker	Tracker mode enter and exit	Pass
	TWM	TWM enter and exit	Pass
	AON ON	AON on under Trancker mode	Pass
Sensor	AON OFF	AON off under Trancker mode	Pass
	Sensor	Accel	Pass
	Sensor	Gyro	Pass
	Sensor	Mag	Pass
	Sensor	ALS/Light	Pass
	Sensor	Pressure	Pass
Connectivity	Sensor	PPG/HRM	Pass
	Co-work	WLAN+BT enable at same time	Pass
	WLAN	wlan off/on 5 times	Pass
		WPA2+AES network connction	Pass
		Ping 1min	Pass
		AP Remenber after off/on	Pass
	BT	Enable/Disable BT 5 times	Pass
		Inquiry devices	Pass
		Pair devices from DUT	Pass
		Unpair devices from DUT	Pass
		DeviceSwitch_Playback_speaker_BT	Pass
		DeviceSwitch_Voice_Call_speaker_BT	Pass
GPS	GPS	GNSS_Position_Fix	Pass
FR	FR56173: Support Dual Camera concurrency	Dual Camera Concurrency -Preview	Pass
		Dual Camera Concurrency -Snapshot	Pass
		Dual Camera Concurrency -Video	Pass

## 9. References

Title	Number
Qualcomm Technologies, Inc.	
<a href="#">SW5100-0/SW5100P-0 Data Sheet</a>	80-24633-1
<a href="#">SW5100-1/SW5100P-1 Data Sheet</a>	80-24634-1
<a href="#">SW5100-1/SW5100P-1 Device Revision Guide</a>	80-24634-4
<a href="#">SW5100-0/SW5100P-0 Pin Assignment and GPIO Configuration Spreadsheet</a>	80-24633-1A
<a href="#">SW5100-1/SW5100P-1 Pin Assignment and GPIO Configuration Spreadsheet</a>	80-24634-1A
<a href="#">SW5100 QFPROM Programming Reference Guide</a>	80-24634-97
<a href="#">SW5100 Android Wearables Software Reference Manual</a>	80-20387-100
<a href="#">SW5100 Android Wearables Software Programming Guide</a>	80-20387-200
<a href="#">SW5100 Android Wearables Quick Start Guide</a>	80-20387-300
<a href="#">SW5100 Clock Plan Reference Manual</a>	80-20387-26
<a href="#">SW5100 Secure Boot Enablement User Guide</a>	80-20387-42
<a href="#">SW5100 Modem Software Technical Overview</a>	80-20387-96
<a href="#">QCC5100 COPROCESSOR SOFTWARE REFERENCE MANUAL</a>	80-20387-600
<a href="#">Qualcomm Product Configuration Assistant Tool (PCAT) User Guide</a>	80-PR518-1
<a href="#">Qualcomm Unified Tools Service (QUTS) User Guide</a>	80-PG522-3
<a href="#">QXDM Professional User Guide</a>	80-V1241-21
<a href="#">QXDM PROFESSIONAL LINUX EDITION USER GUIDE</a>	80-V1241-22
<a href="#">Qualcomm Hexagon Tools Installation Guide</a>	80-N2040-32

## LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

### 1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("Qualcomm Technologies"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "Qualcomm Internal Use Only", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to [qualcomm.support@qti.qualcomm.com](mailto:qualcomm.support@qti.qualcomm.com). This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the *Website Terms of Use* on [www.qualcomm.com](http://www.qualcomm.com), the *Qualcomm Privacy Policy* referenced on [www.qualcomm.com](http://www.qualcomm.com), or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

### 2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.